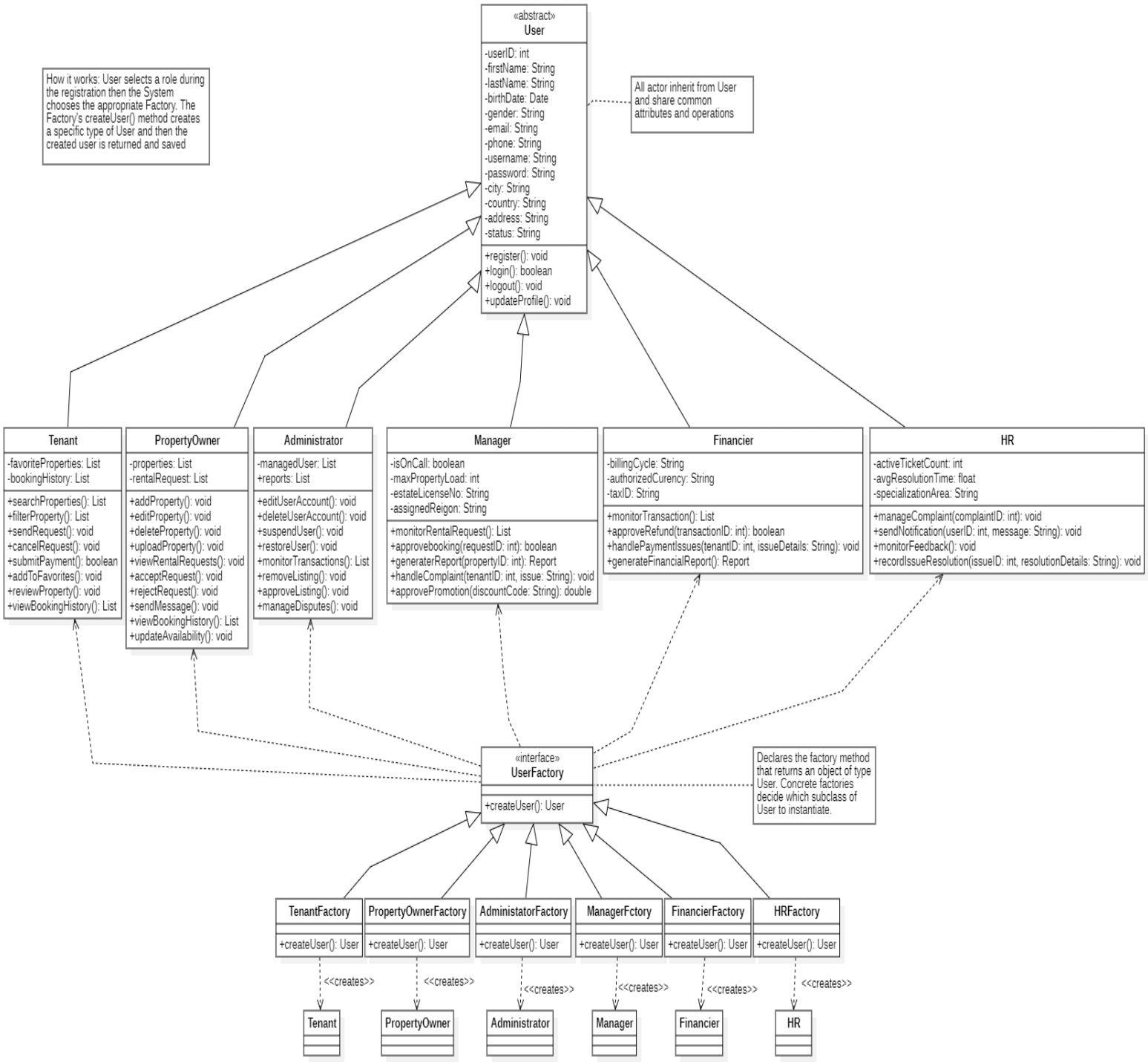
Factory Method Pattern

Justifying the design pattern presence➔It is justified by the functional requirements where the platform must allow all user roles to register (FR41), log in (FR42), and log out (FR43) using a unified process. When a new account is created, the system must instantiate the correct subclass of the abstract User class based on the selected role during registration. The Factory Method pattern centralizes and abstracts this object creation by delegating it to specialized factory classes such as TenantFactory, PropertyOwnerFactory, and AdministratorFactory, each responsible for producing an instance of the appropriate user type. This approach eliminates redundant registration logic for each actor and ensures all user accounts are created consistently, with the right attributes and behaviors required for their roles. As a result, the pattern provides a flexible and maintainable solution for handling user account creation and authentication as specified by FR41, FR42, and FR43.

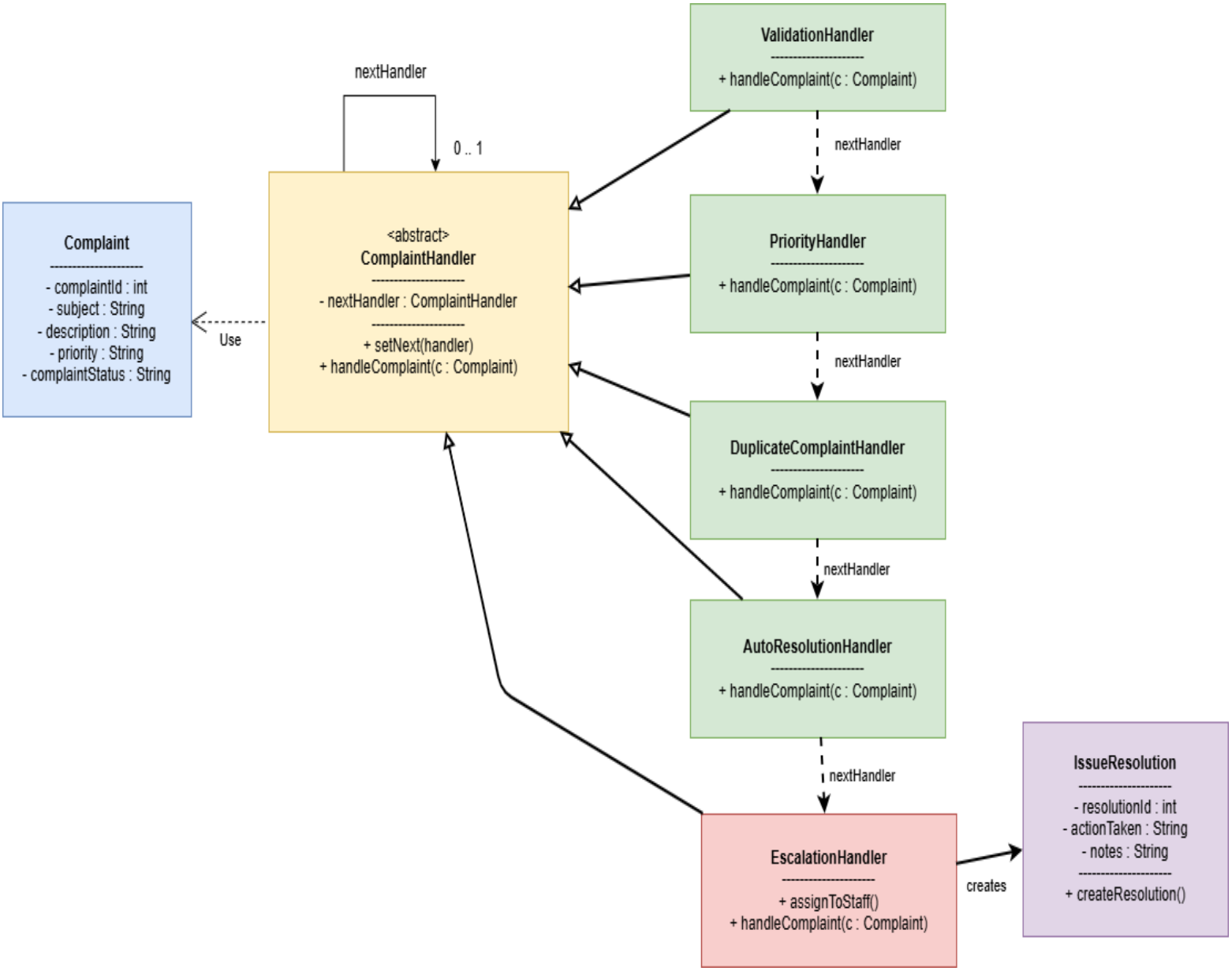


Chain of Responsibility Pattern

Justifying the design pattern presence➔The Chain of Responsibility Pattern is used in the system to automatically process complaints through multiple handling stages without placing all complaint logic in one class.

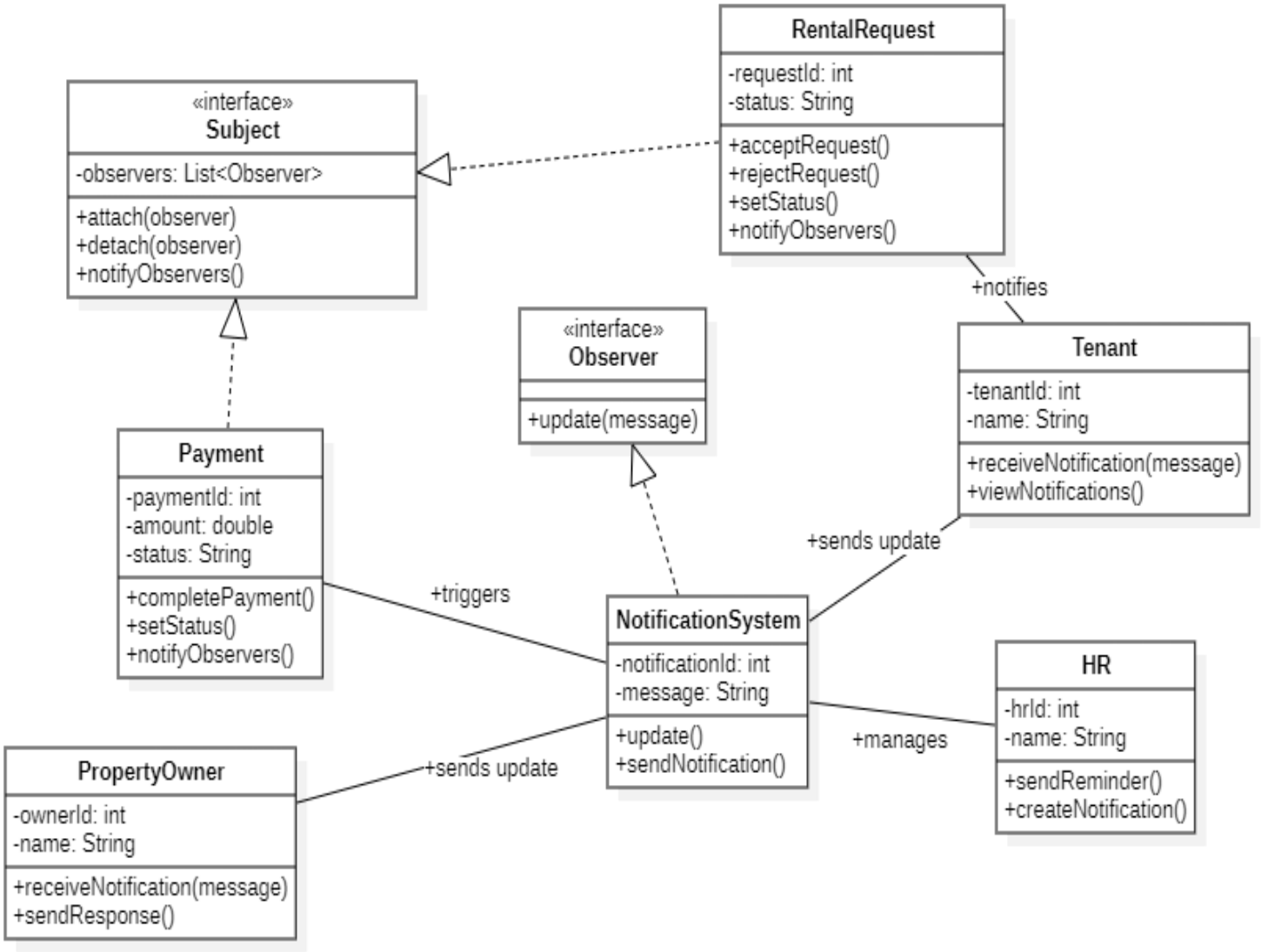
In this design:
ComplaintHandler acts as the abstract handler and stores nextHandler.
ValidationHandler, PriorityHandler, DuplicateComplaintHandler, AutoResolutionHandler, and EscalationHandler act as concrete handlers.Complaint is the request object passed through the chain.IssueResolution stores the final resolution details.

For example:
In FR38, when HR manages complaints, the system first validates and categorizes the complaint automatically.
In FR33, if the complaint cannot be resolved automatically, it is escalated for manager review.
In FR26, unresolved or serious complaints can be escalated to the administrator as a dispute/system issue.



Observer Pattern

Justifying the design pattern presence➔The Observer Pattern is used in the system to automatically notify users when important events occur, such as rental request updates or payment completion.

In this design:
RentalRequest and Payment act as Subjects because their status can change.
NotificationSystem acts as the Observer, listening for these changes.
Tenant and PropertyOwner receive notifications about approvals, rejections, payments, and reminders.

For example:
-In FR17, when a property owner accepts or rejects a rental request, the tenant is notified.
-In FR8 and FR39, payment updates and reminders are also sent automatically.